



StockSense AI

Prototype build plan

A practical roadmap for building a small retail sales forecasting and stock planning application.

TEAM GUIDE / 9 OCTOBER 2026

1

STORE

5

ITEMS

7

FORECAST DAYS

The first working milestone

Choose an item, see its seven-day forecast, enter stock information and receive an explained order suggestion.

Sales history



Seven-day forecast



Order suggestion

Our case study is fictional. Use store 1 and items 1 to 5 from Kaggle's Store Item Demand Forecasting dataset. The selected subset has 9,130 records. Sales history covers 2013 to 2017; stock and supplier inputs are demonstration assumptions.



A simple shop planning tool

One local Streamlit page. Four useful areas. Clear dates and explanations.

01

Sales history

Select item 1 to 5 at store 1 and view recent daily sales. Keep the source item IDs.

02

Seven-day forecast

Show seven dated estimates, a chart and the weekly total. Keep the history cut-off visible.

03

Stock planning

Enter available stock, outstanding units, lead time, review interval and buffer. Explain the calculation.

04

Ask StockSense

Ask about forecasts, orders, history, model performance or help. Retrieve the same dashboard results.

A transparent stock calculation

Coverage: supplier lead time + review interval, up to seven days.

Target stock: forecast sales across coverage + buffer stock.

Inventory position: available stock + outstanding units due within coverage.

Suggested order: round upwards $\max(0, \text{target stock} - \text{inventory position})$.

Illustrative example

10 units forecast per day; lead time 3 days; review interval 1 day; buffer 10 units.

50

TARGET STOCK

40 forecast + 10 buffer

25

INVENTORY POSITION

20 available + 5 incoming

25

SUGGESTED ORDER

50 target - 25 inventory

Assume outstanding units arrive with the next delivery, after the entered lead time. Flag a possible shortage if sales forecast before delivery exceed current stock. Reject negative inputs and unsupported horizons; round only the final order to whole units. The example is arithmetic, not a model result or evidence of savings.



Build in seven stages

Deliver the forecast and stock calculation first, then complete the assistant and experiments.

STAGE AND WORK		EVIDENCE OF COMPLETION
01	Prepare the data Validate columns, dates and quantities. Select the five items, sort histories and reproduce the audit.	9,130 rows; complete daily histories; no duplicate keys.
02	Establish the benchmark Repeat the last week's sales on matching weekdays. This is the seasonal naive baseline.	Seven dated estimates per item and validation errors.
03	Train and compare Add random forest, followed by a small LSTM experiment. Compare identical forecast dates.	Model settings, MAE, RMSE, weekly total error and training time.
04	Calculate stock orders Connect forecasts to stock and delivery inputs. Handle invalid values and arrival assumptions.	At least 10 calculation cases, including pre-delivery shortage risk.
05	Build the dashboard Connect history, forecasts and order planning. Load saved models when the app opens.	All five items work; forecast dates and assumptions are visible.
06	Add the text assistant Train intent recognition. Retrieve calculated results and handle unclear or unsupported questions.	Held-out question results; answers match dashboard quantities.
07	Rehearse and record Run on another laptop. Review the interface, save screenshots and record a backup demonstration.	Repeatable setup, measured response time and review evidence.

Work can overlap: stage 3 starts with random forest. Once it works, continue with the order calculation and dashboard. Finish the LSTM experiment before final evaluation; add the assistant after the first working milestone.



Compare fairly. Record the results.

Use later dates to test forecasts, keeping future sales out of the inputs.

TRAIN

2013-2015

Learn from historical examples

VALIDATE

2016

Choose features and settings

TEST

2017

Measure the fixed methods

Three methods, the same forecast horizon

Weekly baseline

Repeat the most recent week's sales on matching weekdays. Use this benchmark to judge whether learned models add value.

Random forest

One model per item, with seven outputs. Use the preceding 28 daily sales, seven-day and 28-day averages, and target weekday/month. Start with report settings and limit tuning to validation comparisons.

Small LSTM

One 16-unit LSTM and seven-output layer per item. Use 28 days of sales and calendar features, at most 50 epochs and validation-based early stopping. Fit scaling on training data only; fix final training duration before refitting.

Evaluation rules

- Protect the timeline.** Training windows must keep all seven target dates inside training. Later forecasts may use only history observed before their own cut-off.

Freeze the choices. Select on 2016, refit through 2016 and test fixed models on 2017 weeks. Advance seven days; score identical complete weeks and record the dates and counts.
- Measure the outcome.** Report MAE and RMSE by item and overall, seven-day total error and training time. Score the same non-negative predictions shown in the app; keep decimals for accuracy.

Choose honestly. Select the dashboard method using validation results. Retain the baseline if a learned model does not improve it.

Kaggle's 2018 test file has no actual sales. It cannot provide local accuracy scores. Forecasts beyond 31 December 2017 are demonstrations with unverified outcomes. Do not repeatedly tune on the final test year.



Keep each part easy to explain

The assistant retrieves calculated information; the app loads models trained in scripts.

Forecast

Reorder

History

Performance

Help

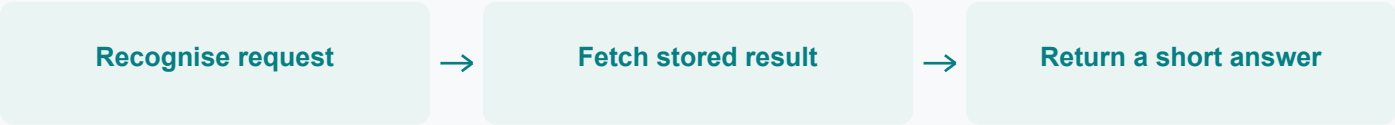
"Forecast for item 2"

Return seven dated estimates and their total from the forecast output.

"Order for item 3?"

Use entered stock inputs. Ask for missing values before calculating an order.

Use TF-IDF with a small intent classifier, checking item numbers separately. Keep different phrasings in training, validation and test sets. Aim for at least **20 correct classifications out of 25 held-out questions**; test unsupported requests separately. Unclear requests receive clarification or help options. Answers must match the dashboard. Use no paid external language service.



Proposed code structure

app.py	Streamlit page
stocksense/	
__init__.py	
data.py	Data validation and history
forecasting.py	Forecasts and model loading
inventory.py	Stock calculation
assistant.py	Intent recognition and answers
scripts/	
train_forecasters.py	Baseline, forest and LSTM
evaluate_forecasters.py	Chronological comparisons
train_assistant.py	Train the intent classifier
tests/	Boundaries and consistency
requirements.txt	Reproducible dependencies
data/	Local CSV files; ignored by Git
artifacts/	Models and results; ignored by Git
docs/	Report, evidence and setup

Core tools: **pandas**, **NumPy**, **scikit-learn** and **Streamlit**. Use Keras with a supported local backend for the LSTM experiment. Check installation and record package versions on the demonstration laptop. Create files as needed rather than adding empty scaffolding.



From working app to presentation

Proposed checkpoints for group agreement. All dates below are in 2026.

16 OCT**Core calculation works**

Data checks, benchmark, first random forest comparison and stock calculation.

23 OCT**The components connect**

Dashboard, LSTM experiment and text assistant connected.

26 OCT**Testing is complete**

Final evaluation, application checks and rehearsal on another laptop.

29 OCT**Evidence becomes deliverables**

Update the report with measured results; prepare the poster and slides.

01 NOV**The group completes its review**

Check signatures, Grammarly evidence and the final PDF layout.

02 NOV**Blackboard submission**

Upload by 23:59. Finish the checks before the deadline.

Agree on owners and reviewers

Use the existing team roles to allocate data, forecasting, dashboard, assistant and testing tasks on GitHub Projects. Give each task an owner and reviewer. One member integrates the parts and checks consistency with the report. Everyone prepares to explain the demonstration.

What we need before calling it complete

- All five items show seven dated forecasts and a correct, explained stock calculation.
- Invalid quantities and unsupported questions are handled clearly; assistant answers match the dashboard.
- Record forecast errors, classifier results, calculation checks, response time and user review outcomes.
- Save three useful screenshots: forecast, order suggestion and assistant response. Rehearse on another laptop.

Keep CSVs and model files out of Git commits. Supply the dataset source, repeatable setup steps and a local demonstration bundle. Save reviewed results in the documentation and keep a backup recording. Update proposed wording to describe the actual implementation before making the poster and slides.